Assignment-4: Defect Management and Test Reporting

Software Testing and Quality Assurance • MCA 2nd Year

Assignment 4 — Answers

Q1. What is a Defect? Explain the importance of defect tracking in software testing.

A defect is a flaw or mistake in a software work product that can cause the product to behave differently from its specified or expected behaviour. A defect may be in requirements, design, code, configuration, or documentation. When it is executed under certain conditions, it may cause a failure visible to a user.

Defect tracking records each defect and follows it through investigation, assignment, correction, retesting, and closure. It is important because it prevents issues from being forgotten, establishes clear ownership, supports prioritisation by severity and business impact, provides a history of decisions, helps measure product quality, and reveals recurring problem areas. It also improves communication between testers, developers, managers, and product owners.

For example, if checkout charges a customer twice, tracking records the steps, evidence, severity, owner, fix, and retest result until the issue is resolved.

Q2. Explain the Defect Life Cycle with a suitable diagram.

The defect life cycle is the sequence of states a defect passes through from discovery to final resolution. A common workflow is shown below; organisations may use different state names.

New → Assigned → Open / In Progress → Fixed → Retest → Verified → Closed
↑ |
■■■ Reopened ■■■■■■■

A tester reports a new defect with reproducible steps and evidence. A triage lead reviews severity and priority and assigns it to a developer. The developer investigates and fixes the issue, then marks it ready for retesting. The tester checks the fix and relevant regression tests. If the expected behaviour is restored, the defect is verified and closed. If it still fails, it is reopened and returned for further work.

Other possible states include Rejected/Invalid, Duplicate, Deferred, and Cannot Reproduce. These help the team handle reports that are not valid, already logged, postponed, or not reproducible.

Q3. What are the different states of a defect in the Defect Life Cycle? Explain each state.

Common defect states are:

1. New: a tester has logged the defect, but it has not yet been reviewed.
2. Assigned: responsibility has been given to a developer or team.
3. Open/In Progress: the defect is accepted and investigation or coding is underway.
4. Fixed/Resolved: a code or configuration change has been made and is ready for verification.
5. Retest: the tester checks whether the fix works in the relevant build.
6. Verified: the tester confirms the reported problem is corrected.
7. Closed: the defect record is formally completed.
8. Reopened: the problem remains or has returned, so further correction is needed.
9. Rejected/Invalid: the behaviour is expected or the report is not a valid defect.
10. Duplicate: the same issue is already recorded elsewhere.
11. Deferred: the team decides to address it in a later release.
12. Cannot Reproduce: the team cannot reproduce the problem using the available details.

Not every project uses all states. Clear state definitions help everyone understand the next action and the defect's current status.

Q4. Explain the process of Defect Tracking and Reporting in software testing.

Defect tracking and reporting begins when a tester observes actual behaviour that differs from the expected result. The tester first checks whether the issue is reproducible and whether a similar defect already exists. A clear report is then created with the environment, build, steps, expected and actual results, severity, priority, and supporting evidence.

The defect is reviewed during triage, where the team confirms validity, assesses impact, sets priority, and assigns an owner. The developer investigates and records progress, root cause if known, and the fix version. The tester retests the corrected build and performs relevant regression tests. If the issue is resolved, the defect is verified and closed; otherwise, it is reopened.

Reports and dashboards summarise open defects, age, severity, trends, and ownership. This allows the team to monitor risks and make informed release decisions.

Q5. What information should be included in a Defect Report? Explain.

A complete defect report should contain:

- Defect ID and title: a unique reference and short, specific description.
- Product/build details: application version, module, platform, browser, device, and environment.
- Preconditions: account, configuration, or data required before testing.
- Steps to reproduce: numbered steps that another person can follow.
- Expected result: the behaviour required by the specification.
- Actual result: what happened, including error messages.
- Severity and priority: technical/business impact and urgency.
- Reproducibility: whether it happens always, sometimes, or once.
- Evidence: screenshots, video, logs, stack traces, or network details.
- Ownership and status: reporter, assignee, date, current state, and related test case.
- Resolution information: fix version, root cause where available, retest result, and closure date.

A report should be objective and concise. Avoid vague titles such as “Not working”; describe the exact feature and failure.

Q6. Differentiate between Defect Severity and Defect Priority with suitable examples.

Severity describes how seriously a defect affects system operation; priority describes how urgently it should be fixed. Severity is mainly about impact, while priority is about scheduling and business need.

- High severity, high priority: payment processing is broken for all users; fix immediately.
- High severity, lower priority: a serious failure in a rarely used legacy feature may be scheduled later if it does not affect current users, subject to risk and compliance.
- Low severity, high priority: a spelling error on the home page during a major public launch may need an urgent correction.
- Low severity, low priority: a minor alignment issue on an internal page may be fixed later.

Severity is commonly assessed by testers and technical stakeholders; priority is often decided through triage with product/business input. The exact decision depends on context and risk.

Q7. Explain the Defect Management Process in detail.

Defect management is the controlled process of identifying, recording, analysing, prioritising, assigning, resolving, verifying, and learning from defects throughout the software life cycle.

1. Detection: identify a mismatch through reviews, static analysis, or test execution.
2. Validation and logging: reproduce the problem, check for duplicates, and create a report with evidence.
3. Triage: confirm validity, assess severity, priority, business impact, and release risk.
4. Assignment: allocate an owner and target release or fix version.
5. Investigation and correction: determine the cause, implement a fix, and document changes.
6. Retesting: verify the correction against the original failure.
7. Regression testing: check that related functionality still works.
8. Closure or reopening: close when the result is correct; reopen if the defect persists.
9. Monitoring and reporting: track open issues, ageing, trends, and release blockers.

10. Prevention and improvement: analyse root causes and improve requirements, coding standards, reviews, automation, or training.

A defined process creates accountability, reduces unresolved risk, and supplies data for improving product and process quality.

Q8. What are the main activities involved in Defect Management? Explain each activity.

The main activities are:

- Defect identification: detect problems during requirement reviews, code reviews, testing, or production monitoring.
- Defect logging: record the issue in a tracking system with steps, expected/actual results, and evidence.
- Classification: categorise the defect by module, type, severity, priority, and source.
- Triage and assignment: confirm validity, decide urgency, and assign an accountable owner.
- Investigation and fixing: find the cause and change the software or configuration.
- Retesting: repeat the original test to confirm the fix.
- Regression testing: test related areas to detect side effects.
- Verification and closure: ensure acceptance criteria are met and complete the record.
- Reporting and metrics: communicate current status, backlog, age, and trends.
- Root-cause analysis and prevention: identify why the defect occurred and introduce improvements.

Together these activities support consistent handling from discovery to prevention.

Q9. Explain the role of Defect Management Tools in software testing.

Defect management tools provide a central system for recording, assigning, tracking, and reporting software defects. Examples include Jira, Bugzilla, Azure DevOps, and similar issue trackers.

Their roles include maintaining a searchable defect repository; assigning owners and due dates; controlling workflow states; storing screenshots, logs, and comments; sending notifications; linking defects to test cases, requirements, builds, and code changes; supporting duplicate detection; and generating dashboards and reports. Access control and audit history can also improve accountability.

These tools help distributed teams collaborate, reduce lost or repeated reports, and provide data for triage and release decisions. A tool does not replace a clear process: the team still needs good report-writing practices, agreed state definitions, and responsible triage.

Q10. What are the important features of a good Defect Tracking Tool? Explain.

A good defect tracking tool should provide:

1. Easy defect creation with configurable fields and templates.
2. Workflow and status management that reflects the team's life cycle.
3. Severity, priority, category, component, version, and assignee fields.
4. Search, filtering, sorting, and duplicate identification.
5. Attachments for screenshots, logs, videos, and other evidence.
6. Notifications and comments for collaboration.
7. Links to requirements, test cases, builds, commits, and releases.
8. Dashboards and reports for backlog, ageing, severity, and trends.
9. Role-based access, audit history, and data protection.
10. Integration with test management, CI/CD, source control, and communication tools.
11. Customisation, scalability, reliability, and reasonable usability.
12. Export and backup capabilities where needed.

The best tool is one that fits the project's workflow and is used consistently by the team.

Q11. Explain how defects are identified, recorded, assigned, fixed, retested, and closed during the Defect Management Process.

First, a tester identifies a defect when actual behaviour differs from expected behaviour. The tester confirms the issue, checks for duplicates, and collects reproducible steps and evidence. Second, the tester records it in the tracking tool, including environment, build, expected/actual results, severity, priority, and related test case.

Third, the triage team reviews the report and assigns it to a suitable developer or team. Fourth, the developer investigates the cause, implements a correction, and updates the record with the fix version. Fifth, the tester obtains the corrected build and repeats the original steps. Relevant regression tests are also run to check for side effects.

If the problem is fixed and acceptance criteria are satisfied, the tester marks it verified and the record is closed. If it persists, the tester reopens it with updated evidence. This cycle continues until resolution or an agreed decision such as deferral.

Q12. What are Test Metrics? Explain their importance in software testing.

Test metrics are measurable values used to evaluate testing progress, coverage, quality, and efficiency. Examples include number of tests planned and executed, pass/fail rates, requirements coverage, defects by severity, defect leakage, and average resolution time.

They are important because they show whether work is progressing according to plan, highlight untested areas, help estimate remaining effort, identify recurring failures, and support evidence-based reporting to stakeholders. Historical metrics can also help improve future estimates and processes.

Metrics should not be used in isolation. For instance, a high test pass rate may look positive while important requirements remain untested. Teams should interpret several measures together and consider complexity, risk, and changes in scope.

Q13. Explain different types of Test Metrics used to measure testing activities.

Common test metrics include:

1. Progress metrics: test cases planned, designed, executed, passed, failed, and blocked.
2. Coverage metrics: percentage of requirements, features, risks, or code covered.
3. Defect metrics: defect count, severity distribution, defect density, open/closed ratio, leakage, and resolution time.
4. Productivity metrics: cases designed or executed over a period, considered alongside their complexity.
5. Efficiency metrics: effort and elapsed time spent on testing, retesting, and defect resolution.
6. Automation metrics: suitable tests automated, automation success rate, execution duration, and maintenance effort.
7. Quality/reliability metrics: production defects, escaped defects, incidents, and failure rate.
8. Schedule metrics: planned versus actual completion and milestone slippage.

The choice of metrics depends on project goals. They should be consistent, meaningful, and difficult to game.

Q14. What is Test Reporting? Explain the importance of test reports in software testing.

Test reporting is the process of communicating testing activities, results, coverage, defects, risks, and completion status to relevant stakeholders. Reports may be produced daily, at the end of a test cycle, or before a release decision.

Test reports are important because they provide a transparent view of what was tested and what remains, summarise passed and failed tests, identify critical open defects, communicate risks and limitations, compare progress with the plan, and support release decisions. They also create a record for audits and future projects.

A useful report is accurate, timely, understandable, and tailored to its audience. Managers may need a concise risk summary, while technical teams may need detailed failure logs and defect links.

Q15. What are the main contents of a Test Report? Explain.

A test report commonly contains:

- Report identification: project, release/build, test cycle, date, and author.
- Objectives and scope: features included and excluded.
- Test environment: devices, operating systems, browsers, tools, and configuration.

• Test execution summary: planned, executed, passed, failed, blocked, and not-run cases.
• Coverage: requirements or features tested and remaining gaps.
• Defect summary: open and closed defects, severity/priority, ageing, and critical issues.
• Results and evidence: key outcomes, logs, screenshots, and links to detailed records.
• Risks and limitations: unresolved issues, unavailable environments, or data constraints.
• Exit criteria: whether the agreed completion conditions have been met.
• Conclusion and recommendation: residual risk and whether release criteria appear satisfied.
• Approvals and next steps: owners, actions, dates, and follow-up testing.

The report should distinguish measured facts from recommendations and clearly disclose untested areas.

Q16. Explain different Test Reporting Techniques used in software testing.

Different reporting techniques serve different audiences and decisions:

1. Summary report: a short overview of scope, execution, major defects, risks, and recommendation.
2. Detailed execution report: case-by-case results, actual versus expected outcomes, and evidence.
3. Dashboard: charts and indicators for pass rate, coverage, defect backlog, and progress, often updated regularly.
4. Defect trend report: shows defects opened, fixed, reopened, and remaining over time.
5. Coverage/traceability report: maps requirements or risks to tests and identifies gaps.
6. Daily/weekly status report: communicates completed work, upcoming tasks, blockers, and schedule status.
7. Release-readiness report: evaluates exit criteria, critical defects, residual risks, and test limitations.
8. Automated reports: generated from test frameworks or management tools and linked to logs and build results.

The format should be clear and accurate. Visuals can reveal trends, but should be accompanied by definitions and context.

Q17. Differentiate between Test Metrics and Test Reports.

Test metrics are individual measurements or calculated indicators, while test reports are structured communications that present and interpret testing information.

• Form: metrics are values such as 85% pass rate; reports combine several values with context and conclusions.
• Purpose: metrics measure progress, coverage, or quality; reports explain status, risks, and implications.
• Frequency: metrics may update continuously; reports are issued at agreed intervals or milestones.
• Detail: a metric may be one number; a report may include scope, environment, defects, limitations, and recommendations.
• Example: “92 of 100 test cases passed” is a metric; a release report explains the 92% result, the failed critical payment cases, and the resulting release risk.

Metrics are often inputs to reports. Both are most useful when definitions are consistent and data is reliable.

Q18. Explain how defect metrics can be used to measure the quality of software.

Defect metrics provide evidence about the types, frequency, and persistence of software problems. Useful measures include defect density (defects relative to a size measure), defects by severity, defect arrival and closure rates, reopen rate, defect age, and defect leakage into production.

A high number of critical defects or a growing backlog near release may indicate significant risk. A high reopen rate may suggest incomplete fixes or unclear requirements. Many escaped production defects may indicate gaps in test coverage, environment realism, or review practices. Falling open-defect counts can be positive when fixes are verified and new defects are not increasing.

Metrics need context: a larger or more heavily tested system may reveal more defects. Quality should therefore be assessed with coverage, severity, user impact, reliability, and trend data together, rather than defect count alone.

Q19. Explain how Test Metrics can be used to measure the effectiveness and efficiency of a testing team.

Effectiveness concerns how well testing finds important defects and covers requirements; efficiency concerns the resources and time used to achieve that result.

Effectiveness can be examined using requirements/risk coverage, critical defects detected before release, defect leakage, and the rate at which important failure paths are discovered. Efficiency can be assessed through cycle time, effort per test cycle, automation execution time, defect turnaround time, and planned-versus-actual effort.

For example, if a team increases high-risk coverage and reduces production defect leakage without an unreasonable rise in effort, its testing process may be improving. If test execution is slow because environments are unstable, blocked-test and cycle-time metrics can reveal the bottleneck.

Metrics should account for test complexity, changing scope, and product risk. Counting cases per tester alone is not a fair measure of performance.

Q20. How can metrics be used to identify problems and improve testing effectiveness? Explain.

Metrics can reveal gaps and bottlenecks when the team compares results with goals and historical trends. Low requirements coverage may indicate missing tests; many blocked cases may point to environment or dependency problems; repeated failures in one module may reveal unstable code; long defect resolution times may indicate ownership or investigation delays; and high production defect leakage may show that important scenarios are not being tested.

To improve effectiveness, the team should: (1) define meaningful metrics and reliable data sources; (2) review dashboards and trends regularly; (3) investigate root causes rather than blame individuals; (4) prioritise high-risk gaps; (5) improve test data, environments, reviews, automation, or training as appropriate; (6) assign owners and target dates; and (7) measure again to see whether the change worked.

Metrics are most useful in a continuous-improvement cycle. They should be interpreted with context and never used as a substitute for technical judgement.